

Introduction:

Ear Trainer is the name of this app, and ear training is what it helps a user learn how to do. Ear training as we mean here is the act of training the ear to recognize changes in pitch by studying scales and harmonies. This app allows users to guess the names of scales, harmonies, or arpeggios chosen at random.

These random organizations of notes are created from a link list, built from an abstracted generic linked list, that contains the 12 tones used in western music listed in chromatic order. The ionian scale is created by following a step pattern along the chromatic scale from the tonic of the key. The modal scales are created by following the ionian scale from the tonic of the mode. The harmonies and arpeggios are built out of the scales.

This project follows the original proposal closely, but completes the requirements for the project without incorporating rhythms. In the future, I hope to incorporate both rhythms and sight reading. My ultimate goal is to train an AI model to read music so users can upload music and practice recognizing the harmonies, scales, and rhythms in the music they are trying to learn.

One alteration from the original proposal is that I did not use a circular linked list. Instead, I simply iterated through the list from the beginning if the scale was not complete by the end of the initial loop. This algorithmic pattern can be seen in "Key" (line 29) when building the ionian scale, and in "Harmony" (line 31) when building the aeolian scale (for minor harmonies taken from aeolian of a different key).

Original Proposal:

For my final project I would like to build an ear training and music theory tool.

I would like to have a circular linked list of the 12 notes in the western scale. This way, starting at any note, you can follow the formula for any given scale (whole steps and half steps) to create a collection of the notes needed for it. If our notes are A, B, C, D, E, F, G, we should be able to travel the path of F, G, A, B, C... If we can do this, we can use formulas to create every modal scale in every western key.

The app will have a learning section and a remembering section. Learning will allow the user to listen to and explore different rhythms, scales, and harmonies. Remembering will be more of a flash card style quiz.

I want to incorporate basic rhythms in 4/4 time, interval training for recognizing how far apart notes are by listening to them, and identifying harmonized scales. A harmony for this app will be two notes that play at the same time, and a scale will be seven notes that play separately.

The things I would really like to figure out how to do are execute an audio file, execute two audio files at once, and if possible, create a visual of a key board where the keys that are currently audible light up. If you have any good directions to point me in, please let me know!

Concept Demonstrated 1	Basic class construction, correct convention, well documented code, and adequate testing.	Class examples: src/AbstractKey.java, src/Key.java, src/NoteList.java Convention/Documentation: throughout Testing: src/JUnitTests.java
Concept Demonstrated 2	The ability for your code to grow as requirements change through the use of abstract classes, polymorphism, and interfaces.	Interfaces: iPlay / iNode - decouple what app does from how it does it. Both can be implemented again in different ways. Polymorphism: "head" as iNode of any type of list (NoteList, Integer)(Rhythms can utilize GenLL). Inheritance: AbstractKey -> Key -> Scale and Harmony - in the future Chords can extend from Key or AbstractKey without interfering with others.
Concept Demonstrated 3	A demonstration of the useful and logical use of abstraction using Generics and Lambda Expressions that will facilitate code reuse in other related projects.	src/NoteList extends src/GenLL They hold reusable: nodeMap (src/GenLL line 53), filterNodes (src/GenLL line 58), and callCounter(src/GenLL line 63)
Concept Demonstrated 4	Code reduction using class hierarchies, helper methods, and dry programming skills.	Scale and Harmony extend Key and reuse Ionian (src/Key line 29) NoteList extends GenLL and reuses traversals listed above in Concept 3. src/Guess wins and losses reuse callCounter (lines 33-42)
Concept Demonstrated 5	The creation and logical use of either a hierarchical data ADT or a Linked List ADT created from scratch. (Whatever makes the most sense for your application)	Generic linked list ADT: src/GenLL Utilizes src/EmptyNode and src/Node which implement src/iNode

Concept Demonstrated 6	A mature well thought out design with MVC and any at least one known design pattern.	MVC - View class, Controller class, and Model is everything else. The controller uses an instance of Guess as model access. Bridge - iPlay (implementation of sound produced for abstractions src/AbstractKey and src/Key) Composite - abstracted generic link list (abstract iNode with two implementations Node and EmptyNode) with recursive patterns (src/GenLL and src/NoteList extends GenLL) Observer - React/Redux (Ai generated frontend GUI with intentional design)
Concept Demonstrated 7	General use of SOLID Design Principles used in construction of the application.	Good examples: S - View class (src/View) O - Scale and Harmony extending Key and add new behavior without adding code to Key (src/Scale and src/Harmony) L - iNode covering Node and EmptyNode means no instance of needed I - iPlay only playback control (src/iPlay) D - AbstractKey has iPlay interface and passes it down (src/AbstractKey)
Possible adjustments		
Extensions		+10 Points - Observer Design Pattern and Composite Design Pattern (Bridge Design Pattern as the one required)
Other concepts demonstrated of greater complexity than the required concepts		+10 Points - Client-Server architecture
Creativity, Originality, and Complexity		+10 Points - creative and original ear training app

Academic Integrity Statement:

Online resources consulted: For Java Media Player - Java docs and Geeks for Geeks (No copy)

Generative AI: Claude used for frontend + ScaleServer.java (for connecting frontend)

Computer science, both academically and professionally, is a collaborative discipline and in future courses you are extremely likely to collaborate on your code. However, when you first start to learn it is vital that you put forth individual effort to master this foundational knowledge.

It is also important to give credit to others when it is due. This includes citing code you may have found online as part of an explanation. **It also includes any and all use of generative AI.**

The following rules are intended to help you get the most out of your education and to clarify the line between honest and dishonest work. The professor reserves the right to ask you to verbally explain the reasoning behind any answer or code that you turn in and to modify your project grade based on your answers. It is vitally important that you turn in work that is your own. Follow the guidelines for academic honesty or we're done.

If you have had a substantive discussion of any homework or programming solution with a classmate, then be sure to cite them in your report. If you are unsure of what constitutes "substantive", then ask us or err on the side of caution. You will not be penalized for working together. You must not copy answers or code from another student either by hand or electronically. Another way to think about it is that you should be talking English with one another, not Java.

The following rules apply to anything you hand in for a grade.

- You may not copy anyone else's code under any circumstances. This includes online sources.
- You may not permit any other student to see any part of your code.
- You may not permit yourself to see any part of another student's code.
- You may not post a public question to any discussion board that contains any part of your code.
- You may consult online resources as part of your coursework, but you may not copy code from online sources. If you get an idea of how to solve a problem from an online source, include a short citation in a code block at the top of your file.

The university's academic integrity policy discusses actions regarded as violations and consequences for students:

[Office of Student Conduct and Conflict Resolution - Academic Integrity Policy](#)

AI Statement

I recognize that generative AI tools are here to stay and a valid tool to use. However, your grade is based on your understanding and effort. **It is not acceptable to use these types of tools without citing its use and using it appropriately.** This means you need to completely understand all code that is part of your project; use adequate testing; and use good design.

In general only use generative AI for explanations, translations, interpretation, practice problems, and syntax checking. Do not use generative AI for solution creation or short cuts of any kind. When in doubt ask!

Use of generative AI without citation does not follow academic integrity expectations. **Turning in code that was completely created by generative AI will be considered academic dishonesty and dealt with by university and college policy.**

Code Files:

Frontend GUI and connection generated with AI as approved by Dr. G.